

Week 06: TanStack Router & TanStack Start

TanStack Router

- Co je routing a proč ho potřebujeme?
- Kdy patří stav do URL?
- Definice rout v TanStack Routeru
- Typová bezpečnost, search params, layouty
- Code splitting

TanStack Start

- Co přináší metaframework?
- SPA režim v TanStack Start
- Fullstack aplikace v TanStack Start

TanStack Query & HeyAPI

- TanStack Query s Routerem i Start

Co je routing a proč ho potřebujeme?

Problém z pohledu uživatele

#1 Pošlete kamarádovi odkaz na konkrétní produkt:

- `https://eshop.com` vs. `https://eshop.com/products/42`

#2 Sdílení stavu aplikace přes URL:

- `https://todo.com/tasks?filter=completed`

#3 Uživatel klikne na tlačítko zpět v prohlížeči a nic se nestane.

URL je **univerzální odkaz** na stav aplikace. Bez routingu o něj přicházíme.

Problém z pohledu vývojáře

Jak řešit více stránek v aplikaci?

```
const App: FC = () => {  
  const [currentPage, setCurrentPage] = useState("homepage");  
  
  return (  
    <div>  
      <Navbar onClick={(page) => setCurrentPage(page)} />  
      {currentPage === "homepage" && <Homepage />}  
      {currentPage === "preferences" && <Preferences />}  
      {currentPage === "products" && <Products />}  
    </div>  
  );  
};
```

Naivní přístup. Nepoužívat!

Co potřebujeme od routeru?

- Synchronizace URL s vykreslovanou stránkou
- Dynamické segmenty (`/products/:id`)
- Search parametry (`?filter=completed`)
- Vnořené routy a layouty
- Code splitting
- Typovou bezpečnost

Řešení: TanStack Router

Proč TanStack Router?

- **100% typově bezpečný** — cesty, parametry, search params
- Vestavěná validace search parametrů (se Zod)
- Automatický code splitting
- Výborné devtools
- Navržený pro moderní React (Suspense, transitions)

Kdy patří stav do URL?

Stav v URL vs. stav v komponentě

Stav v URL	Stav v komponentě
Filtry, řazení, stránkování	Otevřený/zavřený modal
Vybraný tab nebo záložka	Rozpracovaný formulář
ID zobrazovaného detailu	Hover efekty, animace
Vyhledávací dotaz	Dočasné UI stavy

Pravidlo: Pokud chcete, aby uživatel mohl stav **sdílet odkazem** nebo se k němu **vrátit tlačítkem zpět**, patří do URL.

Typy stavu v URL

Path parametry — identifikace zdroje

```
/products/42  
/users/john/settings
```

Search parametry — konfigurace zobrazení

```
/products?category=electronics&sort=price&page=2
```

Path parametry říkají **co** zobrazujeme, search parametry říkají **jak**.

Instalace TanStack Routeru

```
npm install @tanstack/react-router
```

Pro automatické generování rout (doporučeno):

```
npm install -D @tanstack/router-plugin
```

Dva přístupy k definici rout

1. File-based routing (doporučeno)

- Routy se generují automaticky ze struktury souborů
- Méně boilerplate kódu
- Plugin generuje typově bezpečný strom rout

2. Code-based routing

- Routy se definují explicitně v kódu
- Vhodné pro menší projekty nebo specifické potřeby

File-based routing — struktura projektu

```
src/
├── routes/
│   ├── __root.tsx      # Root layout
│   ├── index.tsx       # /
│   ├── about.tsx       # /about
│   └── products/
│       ├── index.tsx    # /products
│       └── $productId.tsx # /products/:productId
├── routeTree.gen.ts     # Automaticky generováno
└── main.tsx
```

Soubory v `routes/` přímo odpovídají URL cestám.

Root route

```
// src/routes/__root.tsx
import { createRootRoute, Outlet } from "@tanstack/react-router";

export const Route = createRootRoute({
  component: RootLayout,
});

function RootLayout() {
  return (
    <div>
      <nav>
        <Link to="/">Domů</Link>
        <Link to="/products">Produkty</Link>
        <Link to="/about">O nás</Link>
      </nav>
      <Outlet />
    </div>
  );
}
```

`Outlet` vykreslí obsah aktuální child routy.

Jednoduchá ruta

```
// src/routes/about.tsx
import { createFileRoute } from "@tanstack/react-router";

export const Route = createFileRoute("/about")({
  component: AboutPage,
});

function AboutPage() {
  return <h1>O nás</h1>;
}
```

`createFileRoute` zajistí typovou bezpečnost — cesta musí odpovídat umístění souboru.

Dynamické routy — path parametry

```
// src/routes/products/$productId.tsx
import { createFileRoute } from "@tanstack/react-router";

export const Route = createFileRoute("/products/$productId")({
  component: ProductDetail,
});

function ProductDetail() {
  const { productId } = Route.useParams();
  //      ^? string – vždy typově bezpečné

  return <h1>Produkt č. {productId}</h1>;
}
```

Prefix `$` označuje dynamický segment. Parametr je vždy `string`.

Vnořené routy a layouty

```
src/routes/  
├── products/  
│   ├── route.tsx      # Layout pro /products/*  
│   ├── index.tsx      # /products  
│   └── $productId.tsx # /products/:productId
```

```
// src/routes/products/route.tsx  
export const Route = createFileRoute("/products")({  
  component: ProductsLayout,  
});  
  
function ProductsLayout() {  
  return (  
    <div>  
      <h1>Produkty</h1>  
      <ProductsNavbar />  
      <Outlet />  
    </div>  
  );  
}
```

Navigace — Link

```
import { Link } from "@tanstack/react-router";

function Navigation() {
  return (
    <nav>
      {/* Statická cesta */}
      <Link to="/about">0 nás</Link>

      {/* Cesta s parametrem – typově bezpečná */}
      <Link to="/products/$productId" params={{ productId: "42" }}>
        Produkt 42
      </Link>

      {/* Aktivní stav – automatická třída */}
      <Link to="/products" activeProps={{ className: "active" }}>
        Produkty
      </Link>
    </nav>
  );
}
```

Na rozdíl od `<a>` tag `<Link>` neprovádí plný reload stránky.

Typově bezpečná navigace

TanStack Router vynucuje správné parametry v `<Link>` :

```
//  Správně
<Link to="/products/$productId" params={{ productId: "42" }}>
  Detail
</Link>

//  TypeScript chyba – chybí params
<Link to="/products/$productId">
  Detail
</Link>

//  TypeScript chyba – neexistující cesta
<Link to="/nonexistent">
  Nikam
</Link>
```

Toto je hlavní výhoda oproti jiným routerům — chyby v odkazech odchytíte při kompilaci.

Search parametry — validace se Zod

```
// src/routes/products/index.tsx
import { z } from "zod";
import { createFileRoute } from "@tanstack/react-router";

const productsSearchSchema = z.object({
  category: z.string().optional(),
  sort: z.enum(["price", "name", "rating"]).optional(),
  page: z.number().default(1),
});

export const Route = createFileRoute("/products/")({
  validateSearch: productsSearchSchema,
  component: ProductList,
});
```

Search parametry jsou automaticky validované a typově bezpečné.

Search parametry — použití v komponentě

```
function ProductList() {  
  const { category, sort, page } = Route.useSearch();  
  //      ^? string | undefined  
  //      ^? "price" | "name" | "rating" | undefined  
  //      ^? number (default 1)  
  
  const navigate = Route.useNavigate();  
  
  return (  
    <div>  
      <button onClick={() => navigate({ search: { page: page + 1 } })}>  
        Další stránka  
      </button>  
  
      <Link to="/products" search={{ category: "electronics", page: 1 }}>  
        Elektronika  
      </Link>  
    </div>  
  );  
}
```

Loader — načítání dat pro routu

```
// src/routes/products/$productId.tsx
export const Route = createFileRoute("/products/$productId")({
  loader: async ({ params }) => {
    const product = await fetchProduct(params.productId);
    return { product };
  },
  component: ProductDetail,
});

function ProductDetail() {
  const { product } = Route.useLoaderData();

  return (
    <div>
      <h1>{product.name}</h1>
      <p>{product.description}</p>
    </div>
  );
}
```

Data se začnou načítat **před vykreslením** komponenty — žádný loading spinner při navigaci

Pending UI

Co se zobrazí, když se data ještě načítají?

```
export const Route = createFileRoute("/products/$productId")({  
  loader: async ({ params }) => fetchProduct(params.productId),  
  pendingComponent: () => <div>Načítání produktu...</div>,  
  errorComponent: ({ error }) => <div>Chyba: {error.message}</div>,  
  component: ProductDetail,  
});
```

Každá routa může definovat:

- `pendingComponent` — zobrazí se během načítání
- `errorComponent` — zobrazí se při chybě

Code splitting

TanStack Router podporuje automatický code splitting s file-based routingem.

Stačí přidat `.lazy.tsx` :

```
src/routes/  
├── products/  
│   ├── $productId.tsx      # loader, validate  
│   └── $productId.lazy.tsx # komponenta (lazy loaded)
```

```
// src/routes/products/$productId.lazy.tsx  
import { createLazyFileRoute } from "@tanstack/react-router";  
  
export const Route = createLazyFileRoute("/products/$productId")({  
  component: ProductDetail,  
});  
  
function ProductDetail() {  
  const { product } = Route.useLoaderData();  
  return <h1>{product.name}</h1>;  
}
```


DevTools

```
import { TanStackRouterDevtools } from "@tanstack/router-devtools";

// V root layoutu:
function RootLayout() {
  return (
    <>
      <Outlet />
      <TanStackRouterDevtools />
    </>
  );
}
```

DevTools ukazují:

- Aktuální stav routeru
- Aktivní routu a parametry
- Search parametry
- Historii navigace

TanStack Router — shrnutí

- **Typová bezpečnost** — cesty, parametry, search params, loader data
- **Search params validate** — Zod schema přímo v definici routy
- **File-based routing** — struktura souborů = struktura rout
- **Loadery** — data se načtou před vykreslením
- **Code splitting** — `.lazy.tsx` soubory
- **DevTools** — snadné debugování

TanStack Start

Co je metaframework?

Router = knihovna pro navigaci v rámci aplikace

Metaframework = kompletní řešení pro tvorbu webových aplikací

Metaframework přidává:

- Server-side rendering (SSR)
- Serverové funkce (API bez separátního backendu)
- Streaming, optimalizace, deployment
- Build systém a dev server

Příklady: Next.js, Remix, Nuxt, SvelteKit, **TanStack Start**

Proč TanStack Start?

- Postavený na TanStack Routeru — sdílí typovou bezpečnost
- **Fullstack typová bezpečnost** — od serveru po klienta
- Používá **Vinxi** (Nitro + Vite) jako server
- Flexibilní — SPA i fullstack režim
- Server functions s RPC voláním

TanStack Start jako SPA

TanStack Start umožňuje vytvářet **čistě klientské SPA** aplikace se všemi výhodami metaframeworku:

- Strukturovaný projekt s file-based routingem
- Dev server s HMR
- Optimalizovaný build
- Možnost kdykoliv přejít na SSR bez přepisu aplikace

```
npx create-start-app my-app
```

Struktura SPA projektu

```
my-app/  
├── app/  
│   ├── routes/  
│   │   ├── _root.tsx  
│   │   ├── index.tsx  
│   │   └── products/  
│   │       ├── index.tsx  
│   │       └── $productId.tsx  
│   ├── router.tsx  
│   └── client.tsx  
├── app.config.ts  
├── package.json  
└── tsconfig.json
```

Konfigurace — app.config.ts

```
// app.config.ts
import { defineConfig } from "@tanstack/react-start/config";
import tsConfigPaths from "vite-tsconfig-paths";

export default defineConfig({
  vite: {
    plugins: [tsConfigPaths()],
  },
});
```


Router setup

```
// app/router.tsx
import { createRouter } from "@tanstack/react-router";
import { routeTree } from "../routeTree.gen";

export function createAppRouter() {
  return createRouter({ routeTree });
}

declare module "@tanstack/react-router" {
  interface Register {
    router: ReturnType<typeof createAppRouter>;
  }
}
```

Deklarace `Register` zajistí globální typovou bezpečnost pro celou aplikaci.

Client entry point

```
// app/client.tsx
import { hydrateRoot } from "react-dom/client";
import { StartClient } from "@tanstack/react-start";
import { createAppRouter } from "../router";

const router = createAppRouter();

hydrateRoot(document, <StartClient router={router} />);
```

Routy ve Start — stejné jako v TanStack Routeru

```
// app/routes/index.tsx
import { createFileRoute } from "@tanstack/react-router";

export const Route = createFileRoute("/")({
  component: HomePage,
});

function HomePage() {
  return (
    <div>
      <h1>Vítejte v naší aplikaci</h1>
      <Link to="/products">Procházet produkty</Link>
    </div>
  );
}
```

Vše, co znáte z TanStack Routeru, funguje i v TanStack Start.

Fullstack aplikace v TanStack Start

Server Functions

Server functions umožňují volat serverový kód přímo z klientských komponent.

```
// app/server/products.ts
import { createServerFn } from "@tanstack/react-start";

export const getProducts = createServerFn({ method: "GET" })
  .handler(async () => {
    const products = await db.product.findMany();
    return products;
  });

export const createProduct = createServerFn({ method: "POST" })
  .validator(z.object({ name: z.string(), price: z.number() }))
  .handler(async ({ data }) => {
    const product = await db.product.create({ data });
    return product;
  });
```

Použití server functions v loaderu

```
// app/routes/products/index.tsx
import { getProducts } from "../../server/products";

export const Route = createFileRoute("/products/")({
  loader: async () => {
    const products = await getProducts();
    return { products };
  },
  component: ProductList,
});

function ProductList() {
  const { products } = Route.useLoaderData();

  return (
    <ul>
      {products.map((p) => (
        <li key={p.id}>
          <Link to="/products/$productId" params={{ productId: p.id }}>
            {p.name}
          </Link>
        </li>
      ))}
    </ul>
  );
}
```

Validace vstupů na serveru

```
import { createServerFn } from "@tanstack/react-start";
import { z } from "zod";

const createProductSchema = z.object({
  name: z.string().min(1),
  price: z.number().positive(),
  description: z.string().optional(),
});

export const createProduct = createServerFn({ method: "POST" })
  .validator(createProductSchema)
  .handler(async ({ data }) => {
    // data je typově bezpečné díky validátoru
    const product = await db.product.create({ data });
    return product;
  });
```

Validace probíhá na serveru — klient nemůže poslat nevalidní data.

Server functions v komponentách

```
function CreateProductForm() {  
  const [name, setName] = useState("");  
  const [price, setPrice] = useState(0);  
  
  const handleSubmit = async (e: FormEvent) => {  
    e.preventDefault();  
    const product = await createProduct({ data: { name, price } });  
    console.log("Vytvořeno:", product);  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input value={name} onChange={(e) => setName(e.target.value)} />  
      <input  
        type="number"  
        value={price}  
        onChange={(e) => setPrice(Number(e.target.value))}  
      />  
      <button type="submit">Vytvořit produkt</button>  
    </form>  
  );  
}
```


SSR v TanStack Start

Při fullstack režimu TanStack Start automaticky:

1. **Vykreslí HTML na serveru** — uživatel vidí obsah ihned
2. **Spustí loadery na serveru** — data jsou součástí prvního HTML
3. **Hydratuje na klientovi** — aplikace se stane interaktivní
4. **Další navigace probíhá na klientovi** — SPA chování

Požadavek → Server vykreslí HTML s daty → Klient hydratuje → SPA navigace

SPA vs. Fullstack v TanStack Start

	SPA režim	Fullstack (SSR)
První načtení	Prázdná stránka + JS	Kompletní HTML
SEO	Špatné	Výborné
Server functions	Ne	Ano
Složitost	Nižší	Vyšší
Deployment	Statický hosting	Node.js server

Začněte s SPA — přejít na fullstack lze kdykoliv bez přepisu.

TanStack Query

Co je TanStack Query?

Knihovna pro správu **serverového stavu** v React aplikacích.

Serverový stav se liší od klientského:

- Pochází z externího zdroje (API, databáze)
- Může být **zastaralý** (stale)
- Sdílí ho více uživatelů
- Potřebuje **cache, revalidaci a synchronizaci**

TanStack Query řeší právě tohle.

```
npm install @tanstack/react-query
```

Základní použití TanStack Query

```
import { useQuery } from "@tanstack/react-query";

function ProductList() {
  const { data, isLoading, error } = useQuery({
    queryKey: ["products"],
    queryFn: () => fetch("/api/products").then((r) => r.json()),
  });

  if (isLoading) return <div>Načítání...</div>;
  if (error) return <div>Chyba: {error.message}</div>;

  return (
    <ul>
      {data.map((p) => (
        <li key={p.id}>{p.name}</li>
      ))}
    </ul>
  );
}
```

Query Key — identifikace dat v cache

```
// Všechny produkty
useQuery({ queryKey: ["products"], queryFn: fetchProducts });

// Produkty s filtrem – jiný klíč = jiná cache
useQuery({
  queryKey: ["products", { category: "electronics" }],
  queryFn: () => fetchProducts({ category: "electronics" }),
});

// Konkrétní produkt
useQuery({
  queryKey: ["products", productId],
  queryFn: () => fetchProduct(productId),
});
```

`queryKey` je pole — TanStack Query automaticky rozlišuje cache podle klíče.

Mutate — změna dat na serveru

```
import { useMutation, useQueryClient } from "@tanstack/react-query";

function CreateProductForm() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    mutationFn: (newProduct: { name: string; price: number }) =>
      fetch("/api/products", {
        method: "POST",
        body: JSON.stringify(newProduct),
      }).then((r) => r.json()),
    onSuccess: () => {
      // Invalidace cache – data se znovu načtou
      queryClient.invalidateQueries({ queryKey: ["products"] });
    },
  });

  return (
    <button onClick={() => mutation.mutate({ name: "Nový", price: 100 })}>
      Vytvořit
    </button>
  );
}
```

TanStack Query + TanStack Router

Jak propojit Query s loaderem routy?

```
// src/routes/products/index.tsx
import { queryOptions } from "@tanstack/react-query";

const productsQueryOptions = queryOptions({
  queryKey: ["products"],
  queryFn: fetchProducts,
});

export const Route = createFileRoute("/products/")({
  loader: ({ context: { queryClient } }) => {
    // ensureQueryData – načte jen pokud nejsou v cache
    return queryClient.ensureQueryData(productsQueryOptions);
  },
  component: ProductList,
});
```

Loader zajistí, že data jsou připravená **před vykreslením** stránky.


```
// app/router.tsx
import { QueryClient } from "@tanstack/react-query";

const queryClient = new QueryClient();

export function createAppRouter() {
  return createRouter({
    routeTree,
    context: { queryClient },
  });
}

// Typová deklaráce kontextu
declare module "@tanstack/react-router" {
  interface Register {
    router: ReturnType<typeof createAppRouter>;
  }
}
```

```
// src/routes/__root.tsx
export const Route = createRootRouteWithContext({
  queryClient: QueryClient;
})
```

Použití v komponentě s useSuspenseQuery

```
function ProductList() {  
  // useSuspenseQuery – data jsou zaručeně dostupná díky loaderu  
  const { data: products } = useSuspenseQuery(productsQueryOptions);  
  
  return (  
    <ul>  
      {products.map((p) => (  
        <li key={p.id}>{p.name}</li>  
      ))}  
    </ul>  
  );  
}
```

Proč `useSuspenseQuery` místo `useQuery` ?

- Loader zaručil, že data jsou v cache
- Nemusíme řešit `isLoading` a `error` stavy
- Komponenta je jednodušší a čistší

TanStack Query + TanStack Start

V TanStack Start lze Query kombinovat se server functions:

```
// app/server/products.ts
export const getProducts = createServerFn({ method: "GET" })
  .handler(async () => {
    return db.product.findMany();
  });

// app/routes/products/index.tsx
const productsQueryOptions = queryOptions({
  queryKey: ["products"],
  queryFn: () => getProducts(),
});

export const Route = createFileRoute("/products/")(
  {
    loader: ({ context: { queryClient } }) =>
      queryClient.ensureQueryData(productsQueryOptions),
    component: ProductList,
  }
);
```

Server function se volá na serveru při SSR, na klientovi při navigaci.

Best practices — TanStack Query

Query Options do samostatných souborů

```
// queries/products.ts
export const productsQueryOptions = queryOptions({
  queryKey: ["products"],
  queryFn: fetchProducts,
});

export const productDetailQueryOptions = (id: string) =>
  queryOptions({
    queryKey: ["products", id],
    queryFn: () => fetchProduct(id),
  });
```

Výhody:

- Sdílení mezi loaderem a komponentou
- Konzistentní query keys
- Snadná invalidace

Best practices — TanStack Query

Stale time a garbage collection

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60, // Data jsou čerstvá 1 minutu
      gcTime: 1000 * 60 * 5, // Cache se drží 5 minut
    },
  },
});
```

- **staleTime** — jak dlouho po načtení jsou data považována za čerstvá
- **gcTime** — jak dlouho se nepoužívaná data drží v cache
- Nastavte **staleTime** podle toho, jak často se data mění

Best practices — loader + Query

Přístup	Kdy použít
Pouze loader	Jednorázové načtení, žádný polling
Pouze Query	Data nepotřebujete v loaderu
Loader + Query	Data při navigaci + cache + revalidace

Doporučený vzor: `ensureQueryData` v loaderu + `useSuspenseQuery` v komponentě.

Loader zajistí data před vykreslením, Query zajistí cache a automatickou revalidaci.

HeyAPI — generování API klientů

Problém: ruční psaní API volání

```
// Ruční přístup – žádná typová bezpečnost
const response = await fetch("/api/products");
const products = await response.json(); // any!

// Musíme ručně definovat typy
type Product = { id: string; name: string; price: number };
const products = (await response.json()) as Product[]; // nebezpečné!
```

Problémy:

- Typy se můžou rozejít s API
- Při změně API se kód nerozpadne při kompilaci
- Duplikace typů mezi frontendem a backendem

Co je HeyAPI?

Nástroj, který z **OpenAPI specifikace** automaticky generuje:

- Typově bezpečný API klient
- TypeScript typy pro všechny endpointy
- Integrace s TanStack Query (query options)

```
npm install -D @hey-api/openapi-ts  
npm install @hey-api/client-fetch
```

OpenAPI specifikace

```
# openapi.yaml
openapi: 3.0.0
paths:
  /api/products:
    get:
      operationId: getProducts
      responses:
        "200":
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: "#/components/schemas/Product"
  /api/products/{id}:
    get:
      operationId: getProduct
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
```

Generování klienta

```
// package.json
{
  "scripts": {
    "generate-api": "openapi-ts"
  }
}
```

```
// openapi-ts.config.ts
import { defineConfig } from "@hey-api/openapi-ts";

export default defineConfig({
  client: "@hey-api/client-fetch",
  input: "http://localhost:3000/api/openapi.json",
  output: "src/api/generated",
  plugins: [
    "@hey-api/typescript",
    "@hey-api/sdk",
    "@tanstack/react-query",
  ],
});
```

```
npm run generate-api
```

Co se vygeneruje?

```
src/api/generated/  
├── index.ts           # Re-exporty  
├── types.gen.ts       # TypeScript typy  
├── sdk.gen.ts         # Funkce pro volání API  
└── @tanstack/  
    └── react-query.gen.ts # Query options pro TanStack Query
```

```
// types.gen.ts – automaticky z OpenAPI schématu  
export type Product = {  
  id: string;  
  name: string;  
  price: number;  
  description?: string;  
};
```

Typy se **nikdy** nerozejdou s API — generují se ze stejného zdroje.

Použití vygenerovaného klienta

```
// Bez HeyAPI – ruční, bez typové bezpečnosti
const res = await fetch("/api/products/42");
const product = (await res.json()) as Product;

// S HeyAPI – typově bezpečné, autocomplete
import { getProduct } from "../api/generated";

const { data, error } = await getProduct({ path: { id: "42" } });
//      ^? Product                                ^? typová kontrola
```

Při změně API stačí znovu vygenerovat klienta — TypeScript ukáže, co je potřeba opravit.

HeyAPI + TanStack Query

Plugin `@tanstack/react-query` generuje hotové query options:

```
import {
  getProductsOptions,
  getProductOptions,
} from "../api/generated/@tanstack/react-query.gen";

function ProductList() {
  const { data: products } = useSuspenseQuery({
    ...getProductsOptions(),
  });

  return (
    <ul>
      {products.map((p) => (
        <li key={p.id}>{p.name}</li>
      ))}
    </ul>
  );
}
```

Query keys, typy i fetch funkce — vše vygenerováno automaticky

HeyAPI + TanStack Query — mutace

```
import {
  createProductMutation,
  getProductsOptions,
} from "../api/generated/@tanstack/react-query.gen";

function CreateProductForm() {
  const queryClient = useQueryClient();

  const mutation = useMutation({
    ...createProductMutation(),
    onSuccess: () => {
      queryClient.invalidateQueries({
        queryKey: getProductsOptions().queryKey,
      });
    },
  });

  const handleSubmit = (data: { name: string; price: number }) => {
    mutation.mutate({ body: data });
    // ^? typově bezpečné – musí odpovídat schématu
  };
}
```

HeyAPI + TanStack Router

Vygenerované query options fungují přímo v loaderech:

```
import { getProductsOptions } from "../api/generated/@tanstack/react-query.gen";

export const Route = createFileRoute("/products/")({
  loader: ({ context: { queryClient } }) =>
    queryClient.ensureQueryData(getProductsOptions()),
  component: ProductList,
});

function ProductList() {
  const { data } = useSuspenseQuery(getProductsOptions());
  // ...
}
```

Žádný ruční kód — loader, cache, typy, query keys — vše pochází z OpenAPI specifikace.

Workflow s HeyAPI

1. Backend definuje/aktualizuje OpenAPI specifikaci
- ↓
2. `npm run generate-api`
- ↓
3. TypeScript ukáže chyby, kde se API změnilo
- ↓
4. Opravíme frontend kód
- ↓
5. Vše je typově konzistentní

Single source of truth — OpenAPI specifikace je jediný zdroj pravdy pro typy.

TanStack Router

- Typově bezpečný routing pro React
- Search params validace se Zod
- File-based routing, loadery, code splitting

TanStack Start

- Metaframework s fullstack typovou bezpečností
- Server functions, SSR, flexibilní režimy

TanStack Query

- Správa serverového stavu — cache, revalidace, invalidace
- `ensureQueryData` v loaderu + `useSuspenseQuery` v komponentě

HeyAPI

Zdroje

- [TanStack Router Docs](#)
- [TanStack Start Docs](#)
- [TanStack Query Docs](#)
- [HeyAPI Docs](#)